الاحتراف الكامل

في مكتبات بايثون لعلم البيانات والتعلّم الآلي

الجزء السادس: مكتبة Matplotlib

الفصول ٧ - ١١: الأمثلة العملية، الأداء، التصحيح، أفضل الممارسات، والمشاريع المصغّرة

مرجع احترافي للمبرمجين والمهندسين وعلماء البيانات

جدول المحتويات

جدول المحتويات 2
الفصل السابع: الأمثلة العملية 4
٧.١ المستوى المبتدئ 4
مثال ١: رسم اتجاه مبيعات بسيط 4
٧.٢ المستوى المتوسط 4
مثال ٢: مقارنة توزيعين عبر Histogram متراكب 4
٧.٣ المستوى الاحترافي 4
مثال ٣: لوحة رسوم متعددة (Dashboard) بشبكة GridSpec 4
٧.٤ مثال من مشروع حقيقي: تقرير جودة بيانات بصري 5
تمارين عملية 5
حلول مقترحة (تلميحات) 5
الفصل الثامن: الأداء 7
٨.١ لماذا يصبح الرسم بطيئًا 7
٨.٢ Downsampling قبل الرسم 7
٨.٣ Blitting للرسوم المتحركة السريعة 7
٨.٤ اختيار Backend مناسب للأداء 7
أفضل الممارسات 7
الأخطاء الشائعة 8
الفصل التاسع: التصحيح 9
٩.١ الرسم الفارغ أو غير الظاهر 9
٩.٢ دعم النصوص العربية في Matplotlib 9
٩.٣ استراتيجية تشخيص عامة 9
الأخطاء الشائعة 9
الأسئلة الشائعة 9
الفصل العاشر: أفضل الممارسات 11
١٠.١ الأسلوب الاحترافي الموصى به 11
١٠.٢ قابلية إعادة الاستخدام عبر دوال الرسم 11
١٠.٣ الاختبار البصري 11
أفضل الممارسات 11
الفصل الحادي عشر: مشاريع مصغّرة 12
مشروع ١: مولّد تقرير رسوم بيانية آلي من بيانات CSV وهمية 12
مشروع ٢: أداة مقارنة بصرية لتوزيعات متعددة 12
مشروع ٣: لوحة KPI بصرية بأربعة مؤشرات 12

تمارين عملية 13
حلول مقترحة (تلميحات) 13
مراجع وروابط رسمية 13
خاتمة Matplotlib 13

الفصل السابع: الأمثلة العملية

٧.١ المستوى المبتدئ

مثال ١: رسم اتجاه مبيعات بسيط

```
import matplotlib.pyplot as plt

months = ['يناير', 'فبراير', 'مارس', 'أبريل']
sales = [120, 150, 90, 200]

fig, ax = plt.subplots()
ax.plot(months, sales, marker='o', color='#2E5395')
ax.set_title('اتجاه المبيعات الشهرية')
ax.set_ylabel('المبيعات (بالآلاف)')
fig.savefig('sales_trend.png', dpi=150)
```

شرح الكود: 'marker='o يضيف نقطة واضحة عند كل قيمة شهرية فوق الخط، مما يسهّل قراءة القيم الدقيقة بصريًا دون الاعتماد فقط على الخط المتصل.

٧.٢ المستوى المتوسط

مثال ٢: مقارنة توزيعين عبر Histogram متراكب

```
import numpy as np

group_a = np.random.normal(50, 10, 1000)
group_b = np.random.normal(65, 12, 1000)

fig, ax = plt.subplots()
ax.hist(group_a, bins=30, alpha=0.6, label='المجموعة أ')
ax.hist(group_b, bins=30, alpha=0.6, label='المجموعة ب')
ax.legend()
ax.set_title('مقارنة توزيعين')
```

شرح الكود: alpha=0.6 يجعل الأعمدة شبه شفافة بحيث يظهر التداخل بين التوزيعين بوضوح بدل أن يحجب أحدهما الآخر بالكامل.

٧.٣ المستوى الاحترافي

مثال ٣: لوحة رسوم متعددة (Dashboard) بشبكة GridSpec

```
fig = plt.figure(figsize=(12, 8))
gs = fig.add_gridspec(2, 2, height_ratios=[2, 1])

ax_main = fig.add_subplot(gs[0, :])   # يمتد عبر العمودين
ax_left = fig.add_subplot(gs[1, 0])
ax_right = fig.add_subplot(gs[1, 1])

x = np.linspace(0, 10, 200)
ax_main.plot(x, np.sin(x))
```

```
ax_main.set_title('المؤشر الرئيسي عبر الزمن')

ax_left.bar(['A', 'B', 'C'], [30, 45, 25])
ax_left.set_title('التوزيع حسب الفئة')

ax_right.pie([40, 35, 25], labels=['X', 'Y', 'Z'])
ax_right.set_title('النسب')

fig.tight_layout()
```

شرح الكود: GridSpec يمنح تحكمًا أدق من subplots() العادية في تخصيص مساحات غير متساوية (هنا الصف العلوي أكبر مرتين من السفلي)، وهو الأسلوب الاحترافي لبناء لوحات معلومات مركّبة من عدة رسوم مختلفة الأنواع والأحجام في شكل واحد متناسق.

٧.٤ مثال من مشروع حقيقي: تقرير جودة بيانات بصري

```
import pandas as pd

df = pd.DataFrame({
    'age': np.concatenate([np.random.normal(35, 8, 950), [150, -5, 200]]),
})

fig, axes = plt.subplots(1, 2, figsize=(11, 4))

axes[0].boxplot(df['age'])
axes[0].set_title('صندوق الالتواء (Box Plot) — كشف القيم الشاذة')

axes[1].hist(df['age'], bins=40)
axes[1].set_title('توزيع الأعمار')
axes[1].axvline(df['age'].median(), color='red', linestyle='--', label='الوسيط')
axes[1].legend()

fig.suptitle('تقرير جودة عمود العمر')
fig.tight_layout()
```

شرح الكود: Box Plot أداة بصرية قياسية لكشف القيم الشاذة بسرعة (النقاط خارج "الشوارب")، ودمجه مع Histogram في رسم واحد يعطي صورة كاملة عن شكل التوزيع ونظافته قبل استخدام العمود في أي تحليل لاحق.

تمارين عملية

١. ارسم رسمًا بمحورين Y مختلفين (Twin Axes) لمقارنة درجة الحرارة والرطوبة على نفس المحور الزمني.

٢. ابنِ لوحة من ٤ رسوم فرعية (٢×٢) تعرض كل واحدة نوعًا مختلفًا من الرسم لنفس البيانات.

٣. أضف Box Plot لثلاث مجموعات بيانات مختلفة على نفس الرسم للمقارنة بينها.

حلول مقترحة (تلميحات)

١. استخدم ax2 = ax.twinx() ثم ارسم على ax2 بلون مختلف مع legend منفصل.

٢. fig, axes = plt.subplots(2, 2) ثم استخدم plot, bar, hist, scatter على كل عنصر من axes.flatten().

٣. ax.boxplot([data1, data2, data3], labels=['أ','ب','ج'])

الفصل الثامن: الأداء

٨.١ لماذا يصبح الرسم بطيئًا

رسم عدد ضخم جدًا من النقاط (ملايين) عبر ax.plot العادية قد يصبح بطيئًا وثقيلاً في الذاكرة، لأن Matplotlib تنشئ كائن Artist منفصل قابل للتفاعل لكل عنصر افتراضيًا. كذلك تحديث الرسم في حلقة (كما في الرسوم المتحركة) عبر إعادة رسم Figure بالكامل في كل إطار مكلف جدًا مقارنة بتحديث بيانات Artist موجود فقط.

٨.٢ Downsampling قبل الرسم

```
# بدل رسم مليون نقطة كاملة
huge_data = np.random.randn(1_000_000)

# أخذ عيّنة ممثّلة كافية بصريًا (النتيجة البصرية شبه متطابقة لأغراض الاستكشاف)
sampled = huge_data[::1000]
fig, ax = plt.subplots()
ax.plot(sampled)
```

٨.٣ Blitting للرسوم المتحركة السريعة

```
fig, ax = plt.subplots()
line, = ax.plot(x, np.sin(x))

def update(frame):
    line.set_ydata(np.sin(x + frame / 10))
    return line,

anim = FuncAnimation(fig, update, frames=100, interval=30, blit=True)
```

blit=True يجعل Matplotlib تُعيد رسم العناصر المتغيرة فقط (هنا الخط) بدل إعادة رسم الشكل بأكمله في كل إطار، مما يسرّع الرسوم المتحركة بشكل ملحوظ خصوصًا مع رسوم فرعية متعددة.

٨.٤ اختيار Backend مناسب للأداء

التوصية	السياق
'backend='Agg غير التفاعلي، أسرع وأخف	توليد مئات الرسوم آليًا في سكربت
%matplotlib inline لرسوم ثابتة سريعة، أو widget للتفاعلية	استكشاف تفاعلي في Jupyter
خفّض العيّنة أولاً، أو استخدم datashader كأداة متخصصة	رسم بيانات ضخمة جدًا (ملايين النقاط)

أفضل الممارسات

- أغلق كل Figure بعد استخدامها (plt.close(fig)) عند توليد رسوم كثيرة في حلقة لتفادي تسرب الذاكرة.
- قلّل عدد النقاط المرسومة فعليًا (Downsampling) عند التعامل مع سلاسل بيانات ضخمة جدًا.
- استخدم blit=True في الرسوم المتحركة التفاعلية لتحسين السلاسة.

الأخطاء الشائعة

- توليد مئات الرسوم في حلقة دون استدعاء plt.close(fig)، مما يستهلك ذاكرة متراكمة تدريجيًا حتى في سكربت واحد طويل.
- رسم بيانات خام ضخمة جدًا دون تخفيض العيّنة أولاً، مما يجعل الرسم بطيئًا وغير مقروء بصريًا في آن واحد.

الفصل التاسع: التصحيح

٩.١ الرسم الفارغ أو غير الظاهر

السبب الشائع	المشكلة
نسيان plt.show() في نهاية السكربت	الرسم لا يظهر إطلاقًا في سكربت خارج Jupyter
الرسم على Axes خطأ في شبكة رسوم فرعية متعددة (خلط ضمني عبر pyplot)	الرسم يظهر فارغًا رغم استدعاء plot
عدم تحديد 'backend='Agg قبل الاستيراد	خطأ في العرض على خادم بلا واجهة رسومية
الخط المستخدم لا يدعم العربية، أو غياب معالجة اتجاه النص (bidi)	النصوص العربية تظهر كمربعات أو معكوسة

٩.٢ دعم النصوص العربية في Matplotlib

```
import matplotlib.pyplot as plt
plt.rcParams['font.family'] = 'Arial'   # أو أي خط يدعم العربية مثبَّت على النظام

# لعرض العربية بالاتجاه الصحيح غالبًا نحتاج مكتبتين إضافيتين:
# pip install arabic-reshaper python-bidi
import arabic_reshaper
from bidi.algorithm import get_display

text = get_display(arabic_reshaper.reshape('عنوان الرسم'))
ax.set_title(text)
```

هذا من أخطر الأخطاء التي يقع فيها المطورون العرب تحديدًا: Matplotlib لا تعالج اتجاه النص العربي (RTL) ولا تشكيل الحروف المتصلة تلقائيًا، لذلك تظهر النصوص العربية أحيانًا منفصلة الحروف أو بترتيب معكوس دون معالجة صريحة عبر arabic_reshaper وpython-bidi.

٩.٣ استراتيجية تشخيص عامة

```
print(fig.axes)          # عرض كل الرسوم الفرعية الموجودة فعليًا في الشكل
print(ax.lines)          # عرض كل الخطوط المرسومة على هذا المحور تحديدًا
print(ax.get_xlim())     # التحقق من نطاق المحور الفعلي
```

الأخطاء الشائعة

- استخدام plt.title() بدل ax.set_title() في رسوم فرعية متعددة، فيُطبَّق العنوان على آخر Axes نشط فقط دون تنبيه.
- عرض نصوص عربية دون arabic_reshaper وpython-bidi، مما ينتج نصًا مشوّهًا في الرسم النهائي.

الأسئلة الشائعة

س: لماذا يظهر الرسم في Jupyter لكن ليس عند تشغيل نفس الكود كسكربت؟

ج: Jupyter يستدعي show() ضمنيًا للخلية الأخيرة، بينما السكربت العادي يحتاج استدعاء plt.show() صراحة في النهاية.

الفصل العاشر: أفضل الممارسات

١٠.١ الأسلوب الاحترافي الموصى به

- استخدم دائمًا الواجهة الكائنية fig, ax = plt.subplots() بدل pyplot الإجرائي في أي كود يتجاوز رسمًا استكشافيًا سريعًا.
- غلّف منطق الرسم المتكرر في دوال مخصصة تقبل ax كمعامل بدل تكرار نفس الكود.
- افصل حساب البيانات عن كود الرسم البصري في وحدات منفصلة.
- التزم بمجموعة ألوان (Color Palette) موحدة عبر كل رسوم المشروع لتناسق بصري احترافي.

```
def plot_trend(ax, x, y, title, color='#2E5395'):
    ax.plot(x, y, color=color, linewidth=2)
    ax.set_title(title)
    ax.grid(True, alpha=0.3)
    return ax

fig, axes = plt.subplots(1, 2, figsize=(10, 4))
plot_trend(axes[0], x, y1, 'المبيعات')
plot_trend(axes[1], x, y2, 'التكلفة')
```

١٠.٢ قابلية إعادة الاستخدام عبر دوال الرسم

في المشاريع الاحترافية، تُكتب دوال رسم عامة تقبل ax ككائن مُمرَّر إليها (Dependency Injection)، بدل إنشاء Figure جديد داخل كل دالة. هذا يمنح مرونة تامة لدمج نتيجة الدالة في أي تخطيط رسوم فرعية لاحقًا دون تعديل الدالة نفسها.

١٠.٣ الاختبار البصري

اختبار الرسوم البيانية آليًا أصعب من اختبار الدوال العادية، لكن يمكن على الأقل التحقق برمجيًا من العناصر الأساسية: عدد الخطوط المرسومة، تسميات المحاور، ونطاقات القيم، دون الحاجة لمقارنة الصورة بكسل ببكسل في كل اختبار.

```
def test_plot_trend_creates_one_line():
    fig, ax = plt.subplots()
    plot_trend(ax, [1,2,3], [4,5,6], 'test')
    assert len(ax.lines) == 1
    assert ax.get_title() == 'test'
    plt.close(fig)
```

أفضل الممارسات

- اكتب دوال رسم عامة تقبل ax كمعامل بدل تكرار كود الرسم.
- التزم بلوحة ألوان موحدة عبر كل رسوم المشروع.
- أغلق كل Figure بعد اختبارها أو حفظها (plt.close(fig)) لتفادي تراكم الذاكرة.

الفصل الحادي عشر: مشاريع مصغّرة

مشروع ١: مولّد تقرير رسوم بيانية آلي من بيانات CSV وهمية

```
import pandas as pd, numpy as np, matplotlib.pyplot as plt

rng = np.random.default_rng(3)
df = pd.DataFrame({
    'date': pd.date_range('2024-01-01', periods=180),
    'revenue': rng.normal(1000, 200, 180).cumsum(),
    'region': rng.choice(['North', 'South'], 180),
})

fig, axes = plt.subplots(2, 1, figsize=(9, 7))

for region, group in df.groupby('region'):
    axes[0].plot(group['date'], group['revenue'], label=region)
axes[0].set_title('الإيراد التراكمي حسب المنطقة')
axes[0].legend()

df.groupby('region')['revenue'].last().plot(kind='bar', ax=axes[1], color=['#2E5395',
'#B54A4A'])
axes[1].set_title('الإيراد النهائي لكل منطقة')

fig.tight_layout()
fig.savefig('revenue_report.png', dpi=200)
plt.close(fig)
```

يطبّق هذا المشروع: التكامل المباشر بين Pandas وMatplotlib عبر df.plot(ax=...)، وrgroupby مع حلقة لرسم عدة خطوط، وحفظ تقرير جاهز آليًا بلا تدخل يدوي.

مشروع ٢: أداة مقارنة بصرية لتوزيعات متعددة

```
def compare_distributions(data_dict: dict, bins: int = 30):
    fig, ax = plt.subplots(figsize=(8, 5))
    for label, values in data_dict.items():
        ax.hist(values, bins=bins, alpha=0.5, label=label, density=True)
    ax.legend()
    ax.set_title('مقارنة التوزيعات')
    return fig, ax

fig, ax = compare_distributions({
    'قبل التحسين': rng.normal(50, 15, 1000),
    'بعد التحسين': rng.normal(65, 8, 1000),
})
```

يطبّق هذا المشروع: دالة رسم عامة قابلة لإعادة الاستخدام تقبل أي عدد من المجموعات عبر قاموس، وdensity=True لجعل المقارنة عادلة بين مجموعات بأحجام مختلفة عبر تطبيع المساحة تحت المنحنى.

مشروع ٣: لوحة KPI بصرية بأربعة مؤشرات

```
fig, axes = plt.subplots(2, 2, figsize=(11, 8))
fig.suptitle('لوحة مؤشرات الأداء الرئيسية', fontsize=16)

axes[0,0].plot(df['date'], df['revenue']); axes[0,0].set_title('الإيراد عبر الزمن')
axes[0,1].hist(df['revenue'], bins=25); axes[0,1].set_title('توزيع الإيراد')
axes[1,0].bar(*np.unique(df['region'], return_counts=True));
axes[1,0].set_title('عدد السجلات لكل منطقة')
axes[1,1].boxplot(df['revenue']); axes[1,1].set_title('كشف القيم الشاذة')

fig.tight_layout()
```

يطبّق هذا المشروع: تجميع أربعة أنواع رسوم مختلفة (خطي، مدرّج، أعمدة، صندوقي) في لوحة واحدة متناسقة عبر GridSpec الضمني في subplots(2,2)، وهو نمط شائع جدًا في تقارير الأعمال الآلية.

تمارين عملية

١. أضف عمود شهر لبيانات مشروع ١ وارسم رسمًا يوضّح متوسط الإيراد الشهري كأعمدة بدل خط تراكمي.

٢. وسّع دالة compare_distributions لتقبل معامل اختياري لعرض خط عمودي عند متوسط كل مجموعة.

٣. أضف رسمًا خامسًا للوحة KPI يعرض نسبة كل منطقة كرسم دائري.

حلول مقترحة (تلميحات)

١.

,'df.assign(month=df['date'].dt.month).groupby('month')['revenue'].mean().plot(kind='bar
(ax=ax

٢. أضف حلقة for بعد الرسم تستدعي ax.axvline(np.mean(values), color=..., linestyle='--') لكل مجموعة.

٣. استخدم fig, axes = plt.subplots(3,2) أو GridSpec لإضافة موضع خامس، ثم ax.pie(...).

مراجع وروابط رسمية

- التوثيق الرسمي: https://matplotlib.org/stable/
- دليل الأداء: https://matplotlib.org/stable/users/explain/performance.html
- الكود المصدري: https://github.com/matplotlib/matplotlib

خاتمة Matplotlib

بهذا نُتم مرجعًا احترافيًا كاملاً لـ Matplotlib. المكتبة التالية هي Plotly، التي تنقلنا من الرسوم الثابتة إلى الرسوم التفاعلية بالكامل، مع مقارنة مباشرة بين الاثنتين حول متى تختار كل واحدة.